

SHARIF UNIVERSITY OF TECHNOLOGY

Department of Electrical Engineering

PROJECT ASSIGNMENT

C++ Circuit Simulation Engine and SDL Interface

INSTRUCTOR / INSTRUCTORS

Dr Hashemi; Dr Arasteh; Dr Shirali

ACADEMIC CONTEXT

Object-Oriented Programming

Contents

1 Electrical foundations and graph model 2

2 Modified nodal analysis 2

3 Transient and nonlinear analysis 4

4 Component commands and validation 9

5 Sources and output requests 12

6 Netlists, files, and console workflow 13

7 Required error handling and deliverables 14

8 Circuit simulator implementation assignment: graphical interface and persistence 14

9 Scope, feature status, and graphical framework 17

10 Signal data and plotted results 19

11 Subcircuits, schematic drawing, and node labels 20

12 AC and phase sweep controls 24

13 Project storage requirements 27

14 Serialization and Cereal 28

15 Networking extension and its limits 31

16 Deliverable organization 34

1 Electrical foundations and graph model

Build an object-oriented simulator for resistors, capacitors, inductors, and electrical sources. Voltage and current obey Ohm's law for a resistor, power is P = V I with a stated sign convention, and Kirchhoff's laws constrain the circuit graph. Nodes have identifiers; components have identifiers, two endpoints, and values. The internal model must maintain connectivity consistently when elements or nodes are modified.

2 Modified nodal analysis

Use the block system [G B; C D] [V; I] = [J; E]. With n non-ground nodes and m independent ideal voltage sources, G is n by n, B is n by m, C is m by n, and D is m by m. V contains node voltages, I contains auxiliary source currents, J contains current injections, and E contains source voltages. For independent voltage sources, C = B^T and D is zero. Controlled sources can change this simple block structure. A resistor contributes conductance to endpoint diagonals and its negative to the corresponding off-diagonal terms. Current sources contribute to J with consistent direction signs. A voltage source adds an auxiliary current and a voltage constraint through B and C. Explain stamping rather than treating the matrix as an unexplained library input. Gaussian elimination and LU factorization are candidate direct solvers. Row pivoting and singularity checks are necessary. LU factors can be reused for different right-hand sides only while the matrix is unchanged; altered components, frequency, timestep, or nonlinear linearization may require refactorization.



Figure 1: Diagram or recorded result: Modified nodal analysis.

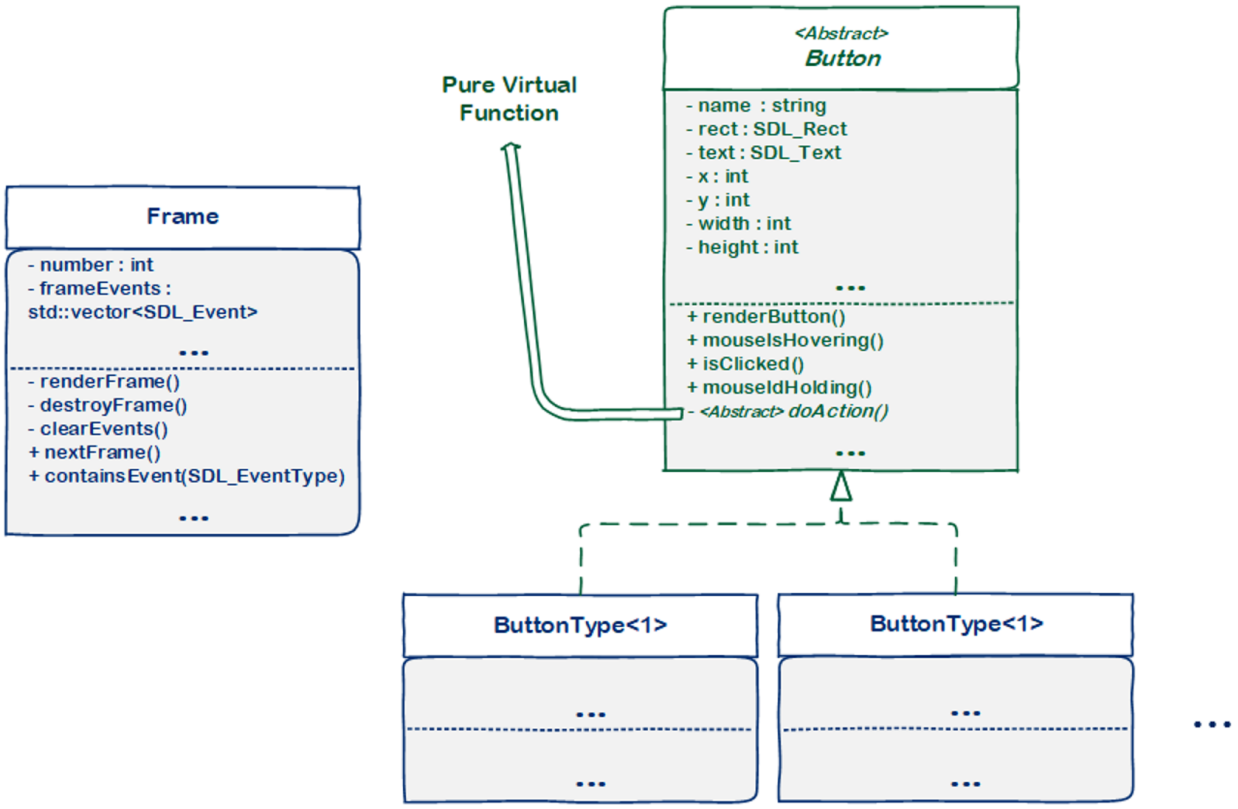


Figure 2: Diagram or recorded result: Modified nodal analysis.

3 Transient and nonlinear analysis

Capacitor and inductor companion models depend on the timestep and previous state. Implement the requested transient path, explaining the discretization, initial conditions, and state updates. Backward Euler and trapezoidal integration are discussed in the source. Adaptive steps also change the companion coefficients and may invalidate cached factors. For a diode, use a nonlinear model and Newton linearization rather than assuming every component is a fixed resistor. The worked example uses a 5-V source, 1-kilohm resistance, saturation current 1e-14 A, thermal voltage 0.026 V, ideality factor 1, and an initial voltage of 0.7 V. Its scalar residual is f(V) = (5-V)/R - I_s(exp(V/V_T)-1), with derivative -1/R - (I_s/V_T)exp(V/V_T). Explain convergence tolerance and iteration limits. The RC worked example uses R = 1 kilohm, C = 1 microfarad, a 5-V step, zero initial voltage, and h = 1 microsecond. Backward Euler gives A = 1/R+C/h = 1.001 S, b = 0.005 A, and the first capacitor voltage approximately 0.004995 V. The RLC example uses R = 1 kilohm, L = 1 millihenry, C = 1 microfarad and the same step/timestep. Its supplied first-step system is [1,-1; -1,-2000] [V;I] = [0;-5], giving approximately 0.00249875 V and 0.00249875 A. The numerical equality of those values depends on the example's coefficients and is not a general physical identity.

Table 1: Transient and nonlinear analysis - numerical data and parameters.

Block	Dimensions	Construction
G	n x n	Conductance stamps between non-reference nodes
B	n x m	Voltage-source incidence
C	m x n	B transpose for independent ideal sources
D	m x m	Zero for independent ideal voltage sources
J	n x 1	Injected independent source currents
E	m x 1	Independent source voltages

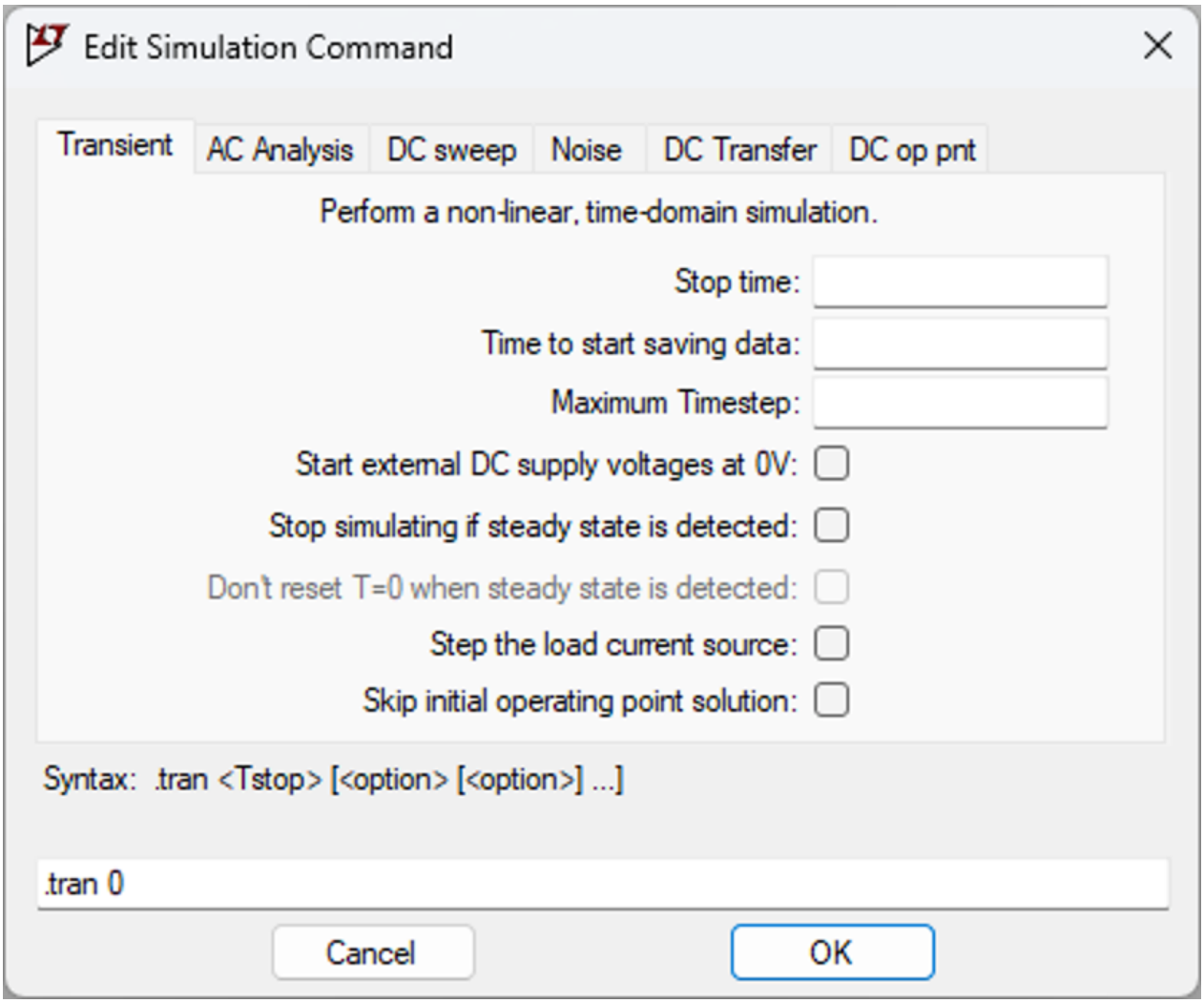


Figure 3: Diagram or recorded result: Transient and nonlinear analysis.

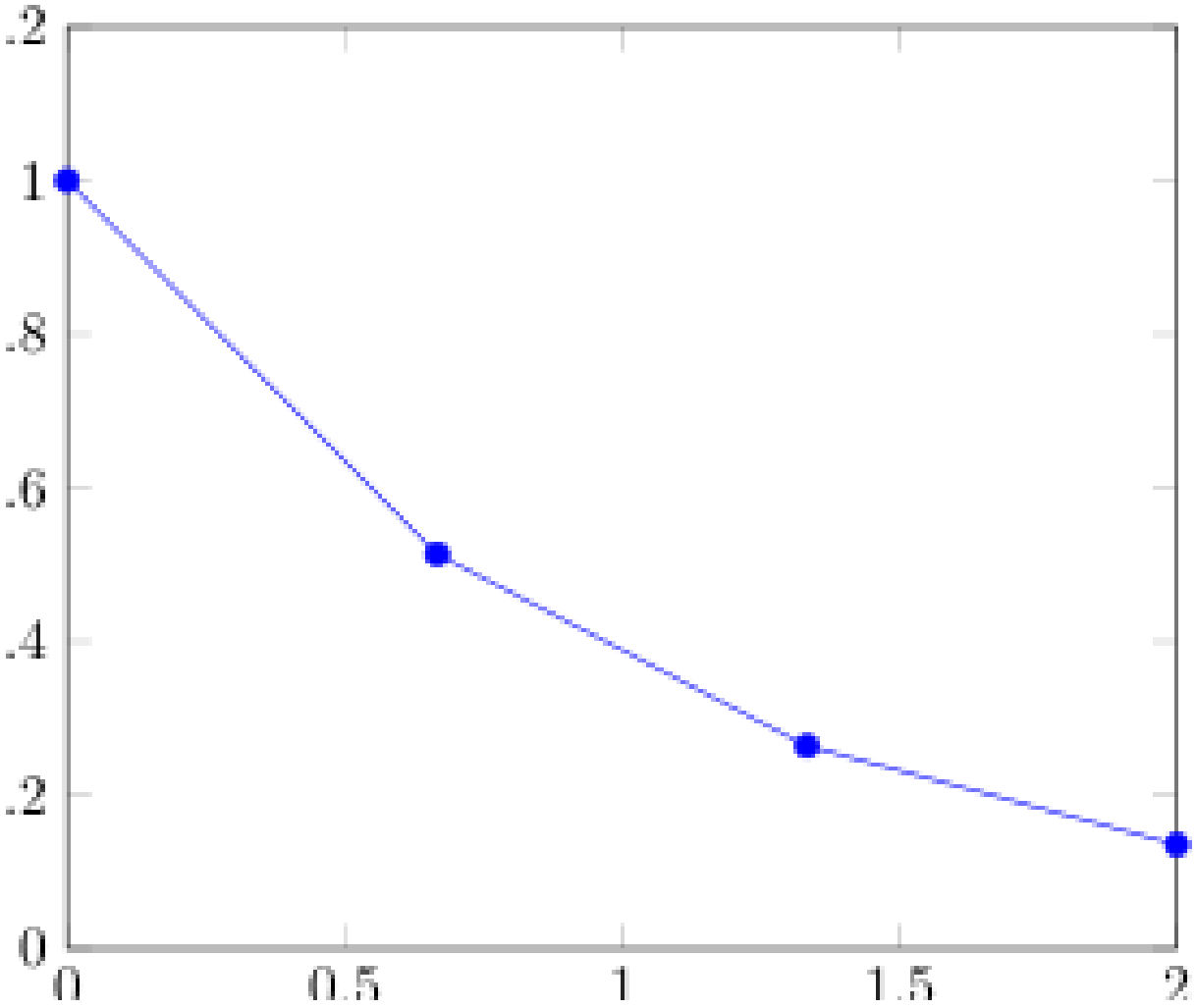


Figure 4: Diagram or recorded result: Transient and nonlinear analysis.

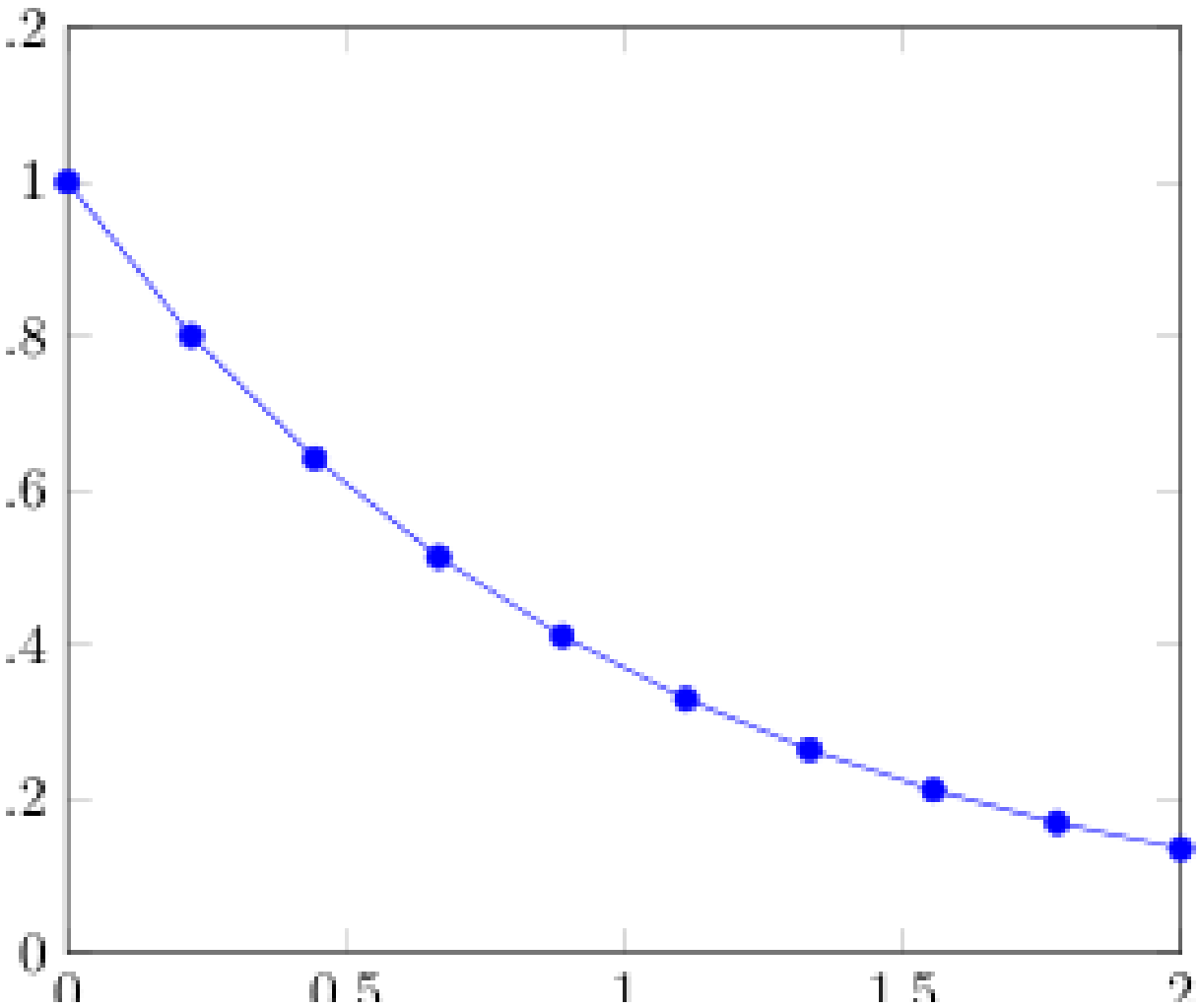


Figure 5: Diagram or recorded result: Transient and nonlinear analysis.

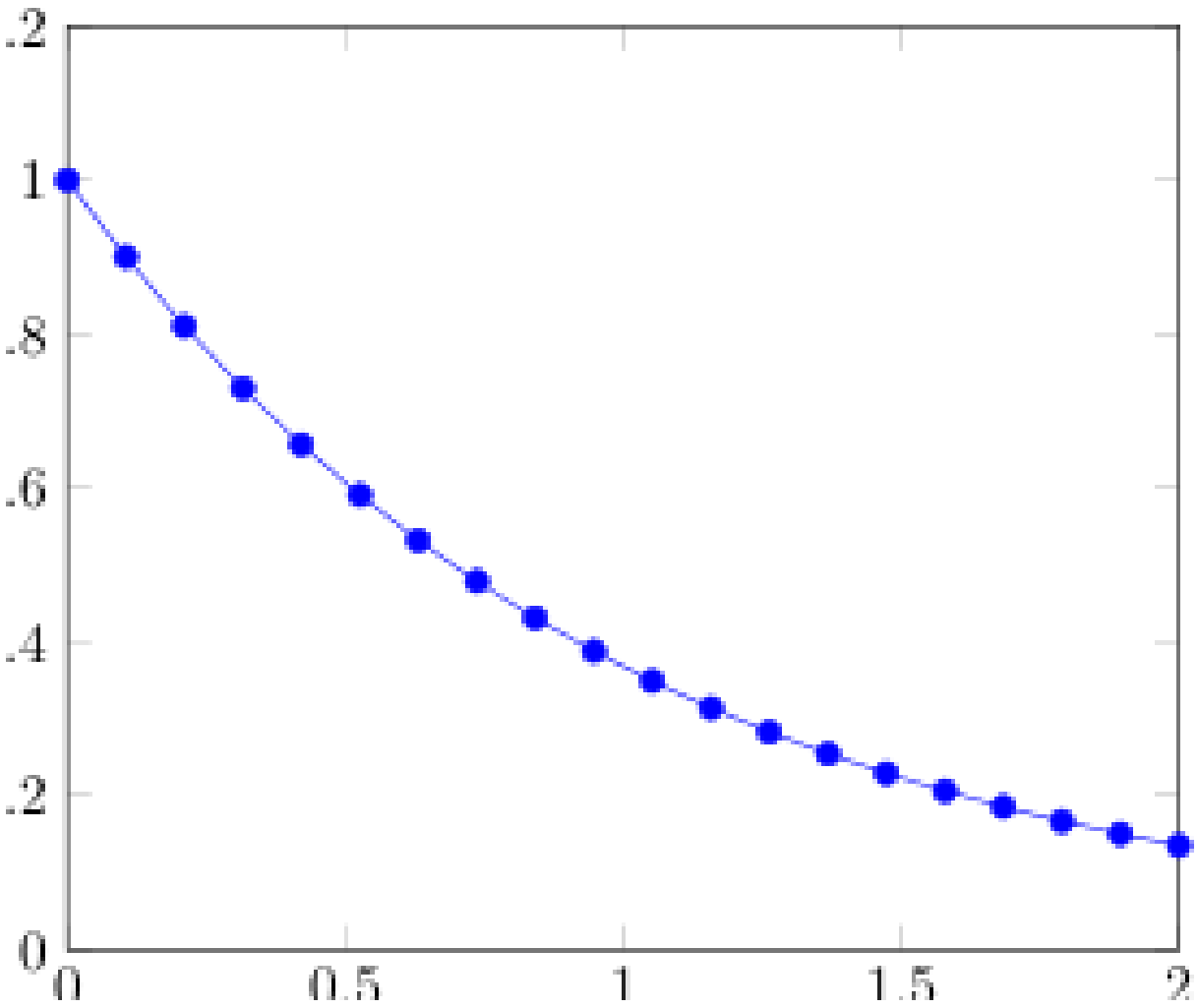


Figure 6: Diagram or recorded result: Transient and nonlinear analysis.

4 Component commands and validation

Support add and delete commands for resistors, capacitors, and inductors. Examples include `add R1 VIN GND 2.2k`, `add C1 VIN GND 1u`, and `add L1 VIN GND 1m`. Identifiers use the corresponding uppercase component prefix. Reject zero or negative passive values, malformed syntax, duplicate names, and deletion of unknown elements. Messages should identify the failed component or argument. Parse bare resistance in ohms and the requested kilo/mega prefixes; capacitance in farads with micro/nano prefixes; inductance in henries with milli/micro prefixes. Scientific notation such as 2e3 or 1e-6 must be handled consistently. Document case sensitivity so that a prefix cannot silently change a value by orders of magnitude. The optional diode command creates `D<name>` between two nodes with the specified D or Z model. Reject unsupported model identifiers and preserve the same duplicate/missing-element checks. A diode class alone does not satisfy the nonlinear-analysis requirement.

`add GND <node>` establishes the reference and `delete GND <node>` removes it according to the prescribed interface. Diagnose a missing node and circuits without a valid reference. `nodes` lists node identifiers. `list` lists elements, and a component-type filter limits the output. `rename node <old> <new>` must reject collisions and missing originals and update every connected endpoint reference.

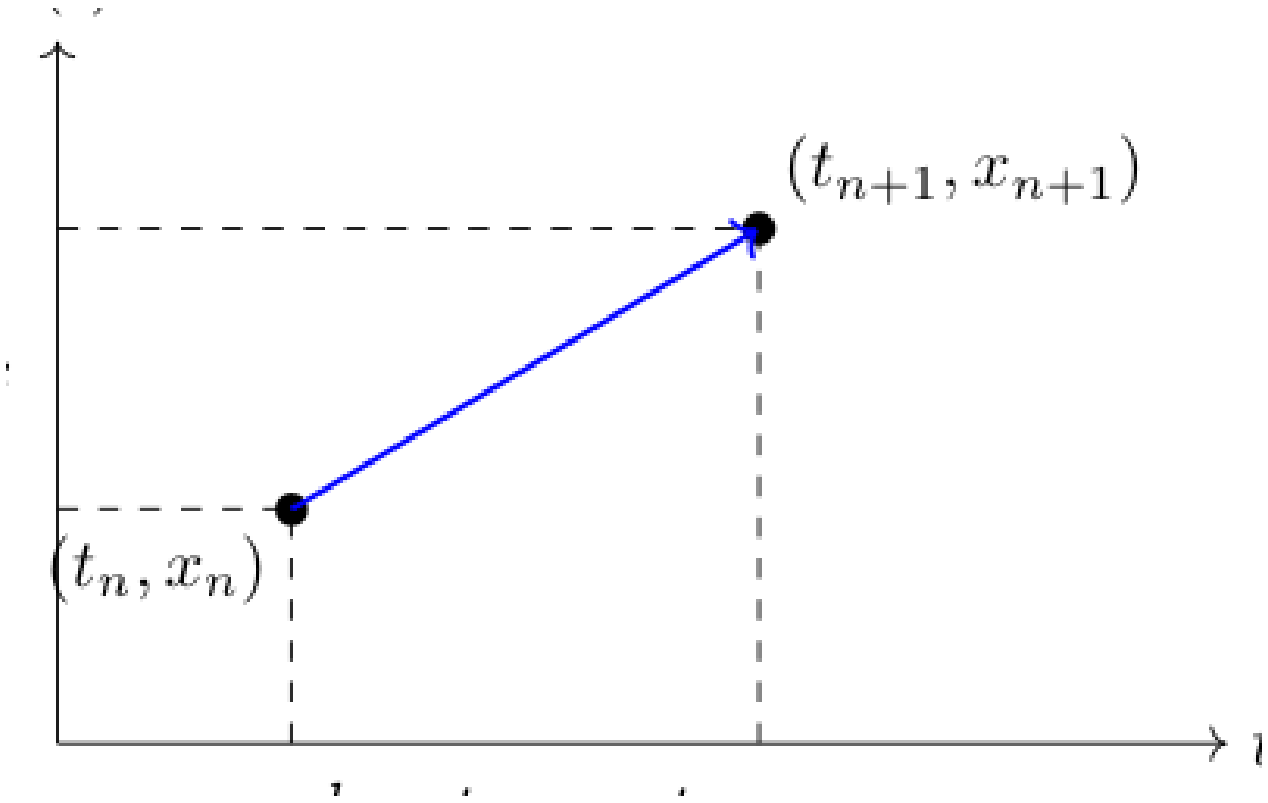


Figure 7: Diagram or recorded result: Component commands and validation.

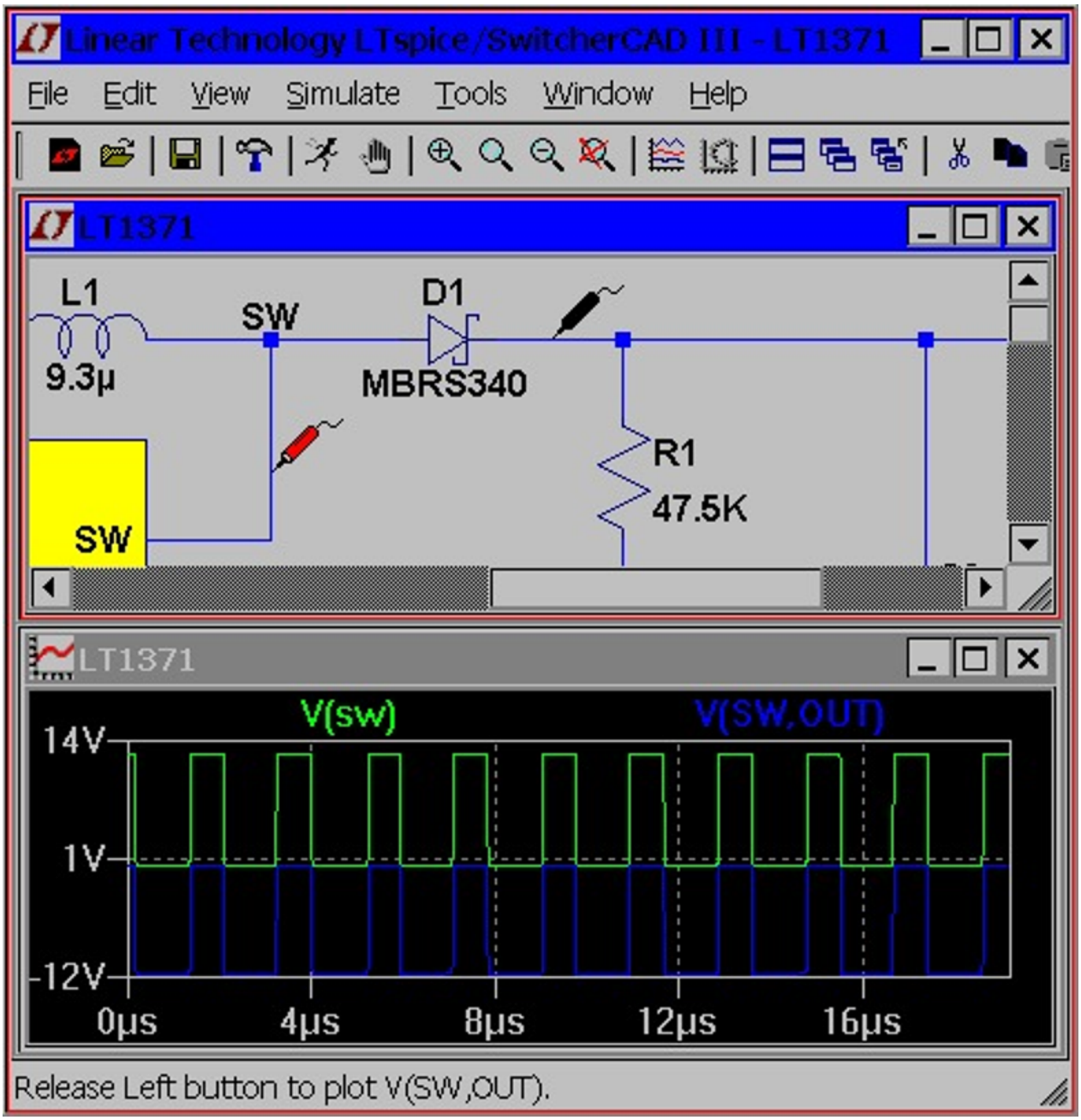


Figure 8: Diagram or recorded result: Component commands and validation.

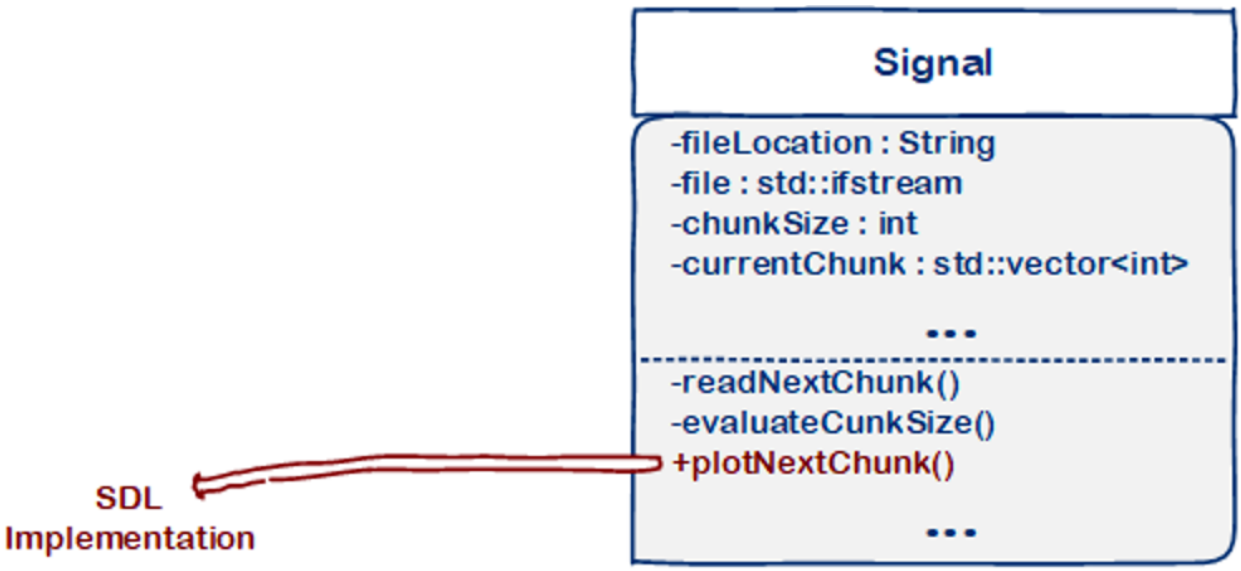


Figure 9: Diagram or recorded result: Component commands and validation.

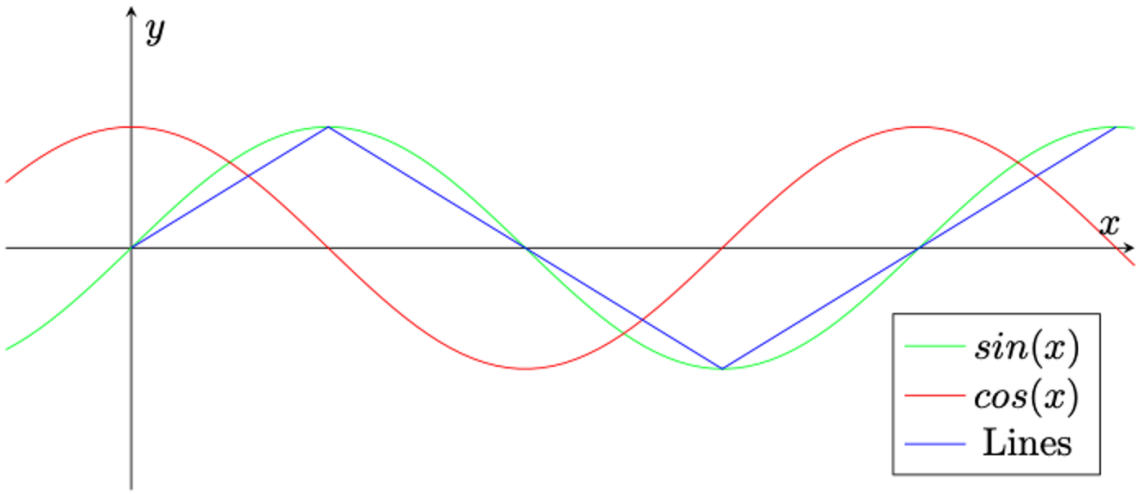


Figure 10: Diagram or recorded result: Component commands and validation.

5 Sources and output requests

Create independent voltage and current sources with names, endpoints, and values. Sinusoidal voltage-source syntax includes offset, amplitude, and frequency. The optional PULSE source includes initial/final values, delay, rise/fall times, width, and period; state the accepted punctuation and units explicitly. Controlled-source types are E for voltage-controlled voltage source, G for voltage-controlled current source, H for current-controlled voltage source, and F for current-controlled current source. Voltage-controlled devices refer to control endpoints; current-controlled devices refer to a named control source. Validate those references before assembling equations. Transient requests include TRAN with timestep, stop time, and the specified optional start/max-step parameters. Requested output variables can include V(node) and I(element), with more than one variable in a print request. Optional DC sweeping specifies a source, start, stop, increment, and requested outputs. Define consistent sampling and column order. Missing analysis parameters, invalid timesteps, unknown variables, or undefined controls are errors rather than empty successful results.

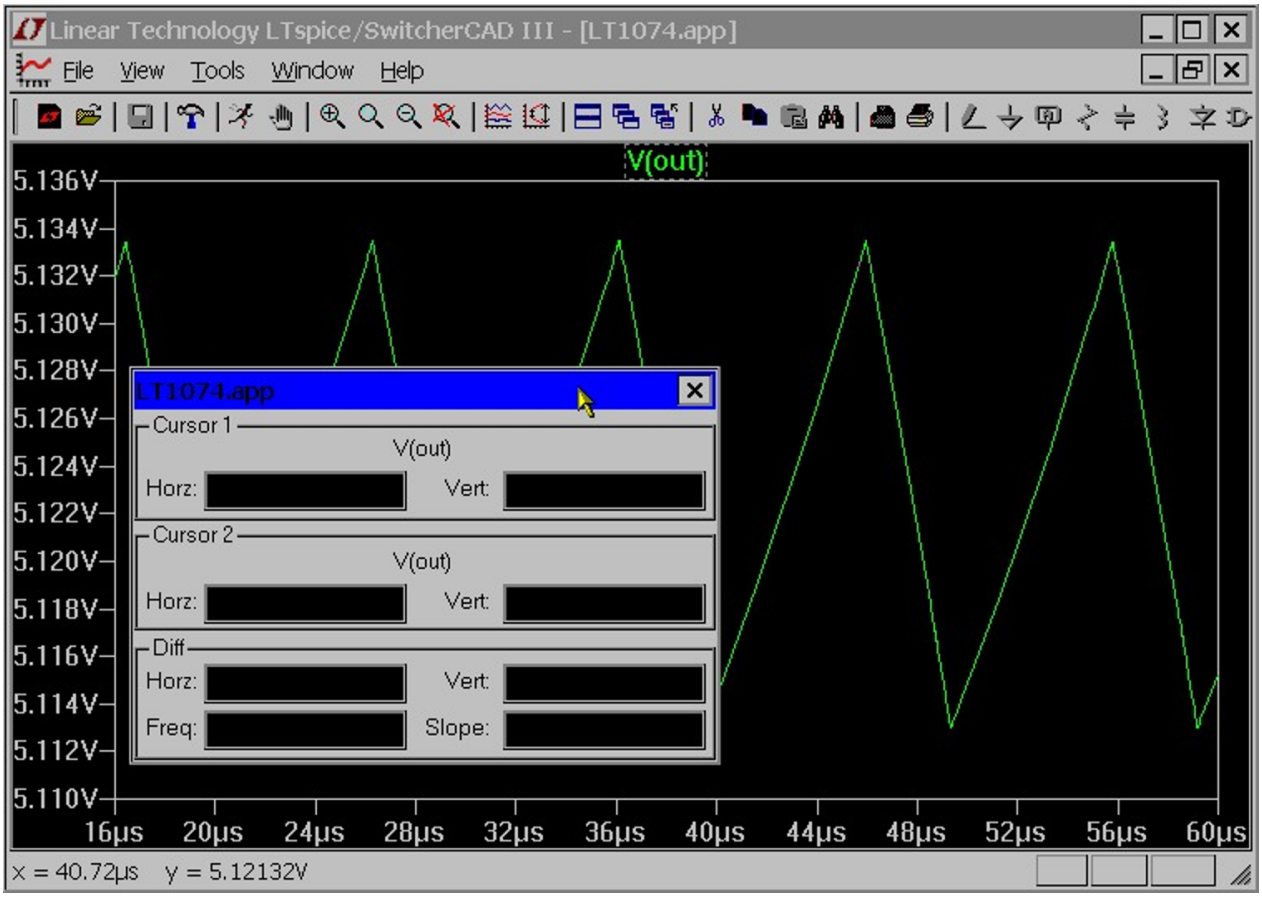


Figure 11: Diagram or recorded result: Sources and output requests.

6 Netlists, files, and console workflow

Support the assignment's netlist representation as well as interactive commands. The simple divider example describes a 5-V source and 1000/2000-ohm series resistors. The supplied more advanced example includes a sinusoidal source, resistor, capacitor, transient/AC directives, and an end marker. Present the existing schematic files in a selection menu. Invalid selection produces the prescribed error and returns to a usable state. `NewFile <file_path>` creates a new file through the documented workflow. File parsing, interactive input, and graphical extensions should share the same validated circuit model.

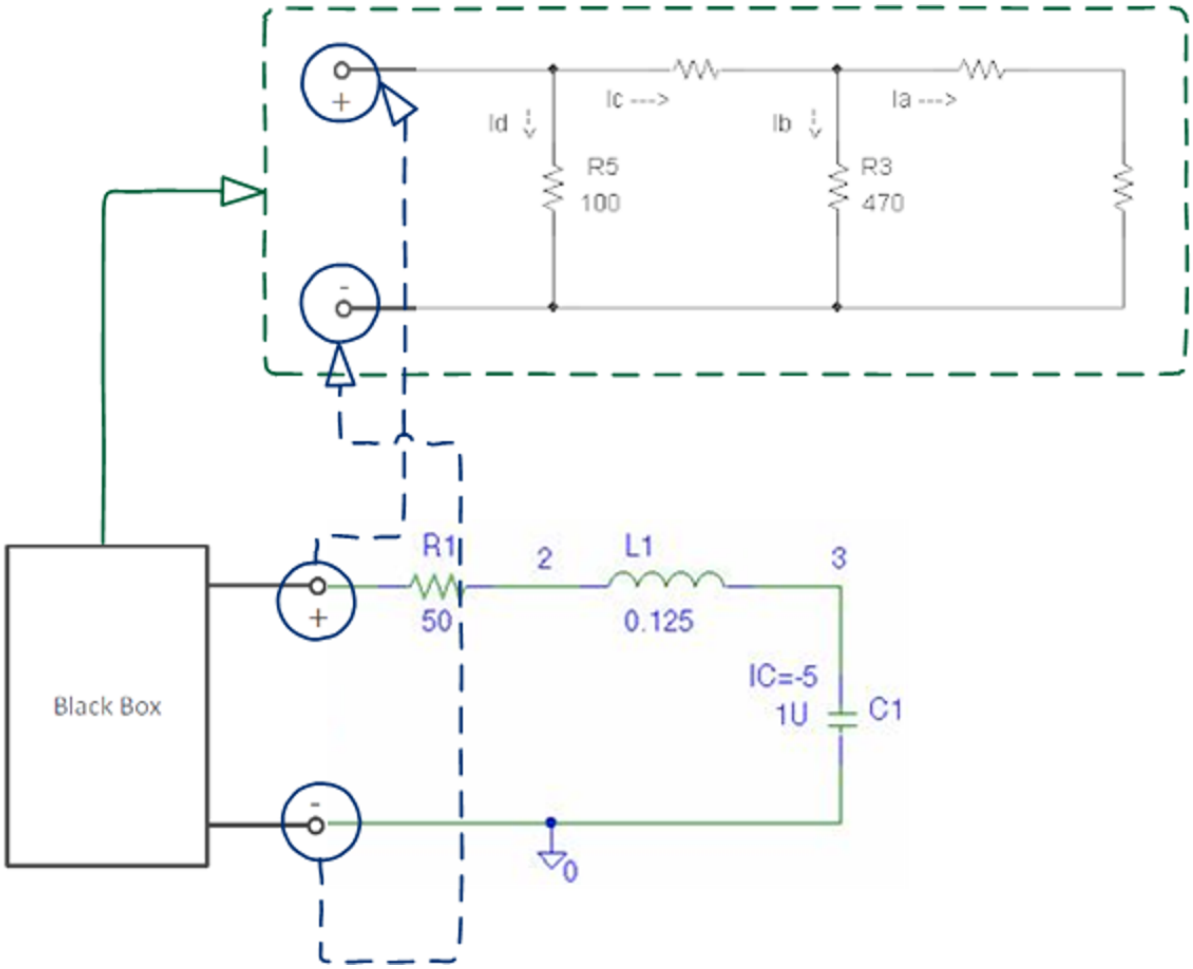


Figure 12: Diagram or recorded result: Netlists, files, and console workflow.

7 Required error handling and deliverables

Diagnose missing ground, floating nodes/subcircuits, contradictory ideal-source constraints, non-positive passive values, undefined controls, duplicate identifiers, malformed commands, and missing simulation parameters. A disconnected drawing is not necessarily invalid if each electrical subcircuit has a valid reference; explain the actual rank/connectivity issue. Submit the C++ implementation and the requested results with readable object-oriented structure. The mandatory transient implementation and optional features must remain distinguishable.

8 Circuit simulator implementation assignment: graphical interface and persistence

```python
import numpy as np
R = 1000
L = 1e-3
C = 1e-6
h = 1e-6
V1 = 5
V_node1_old = 0
I_L_old = 0
A = np.array([[C/h, -1],
              [-1, -R - L/h]])
b = np.array([C/h * V_node1_old,
              -V1 - (L/h)*I_L_old])
x = np.linalg.solve(A, b)
V_node1_new = x[0]
I_L_new = x[1]
print(f"V_node1 at next timestep: {V_node1_new:.6f} V")
print(f"I_L at next timestep: {I_L_new:.6f} A")
```

Figure 13: Implementation and output: Required error handling and deliverables.

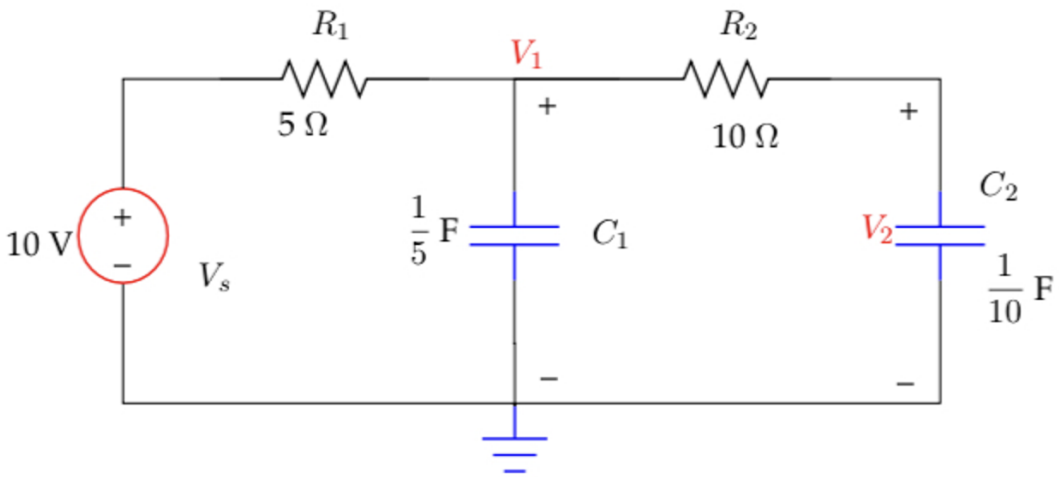


Figure 14: Diagram or recorded result: Required error handling and deliverables.

```
add R R1 0 1 5
add R R2 1 2 10
add C C1 1 3 0.2
add C C2 2 3 0.1
add GND 3
add VoltageSource VS 0 3 10
```

Figure 15: Diagram or recorded result: Required error handling and deliverables.

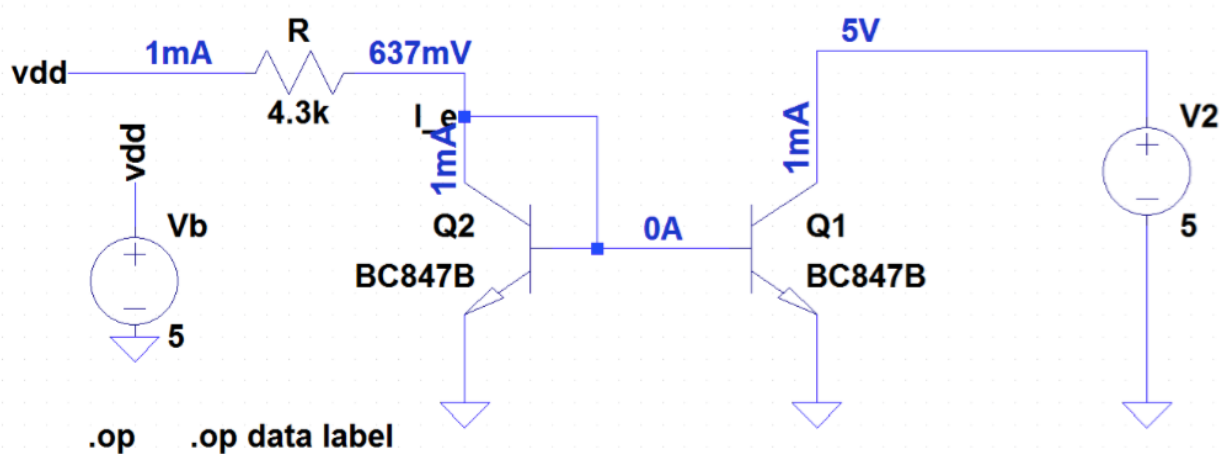


Figure 16: Diagram or recorded result: Required error handling and deliverables.

Table 2: Required error handling and deliverables - numerical data and parameters.

Symbol	Meaning	Unit
omega_start	Lower angular-frequency bound	rad/s
omega_stop	Upper angular-frequency bound	rad/s
N	Number of steps	dimensionless

Table 3: Required error handling and deliverables - numerical data and parameters.

Symbol	Meaning	Unit
phi_start	Lower phase bound	rad
phi_stop	Upper phase bound	rad
omega_base	Base angular frequency	rad/s
N	Number of steps	dimensionless

9 Scope, feature status, and graphical framework

Extend the numerical/console engine with an object-oriented SDL2 graphical interface. The assignment uses colored boxes: red-box features are bonus work for two-person teams and mandatory for three-person teams; purple-box features are bonus work for three-person teams; yellow boxes provide guidance. Use an event loop, window/renderer abstractions, and a reasonable frame rate. Handle input and redraw without blocking the interface with long numerical work. The assignment points to SDL resources and examples; document the development libraries and assets required by the chosen build.

Table 4: Scope, feature status, and graphical framework - numerical data and parameters.

Problem	**Equation**	**Numerical approach**
Linear algebraic	A x = b	Pivoted LU or sparse direct solver
First-order differential	dy/dt = f(t,y)	Backward Euler or trapezoidal integration
Nonlinear	f(x) = 0	Newton-Raphson

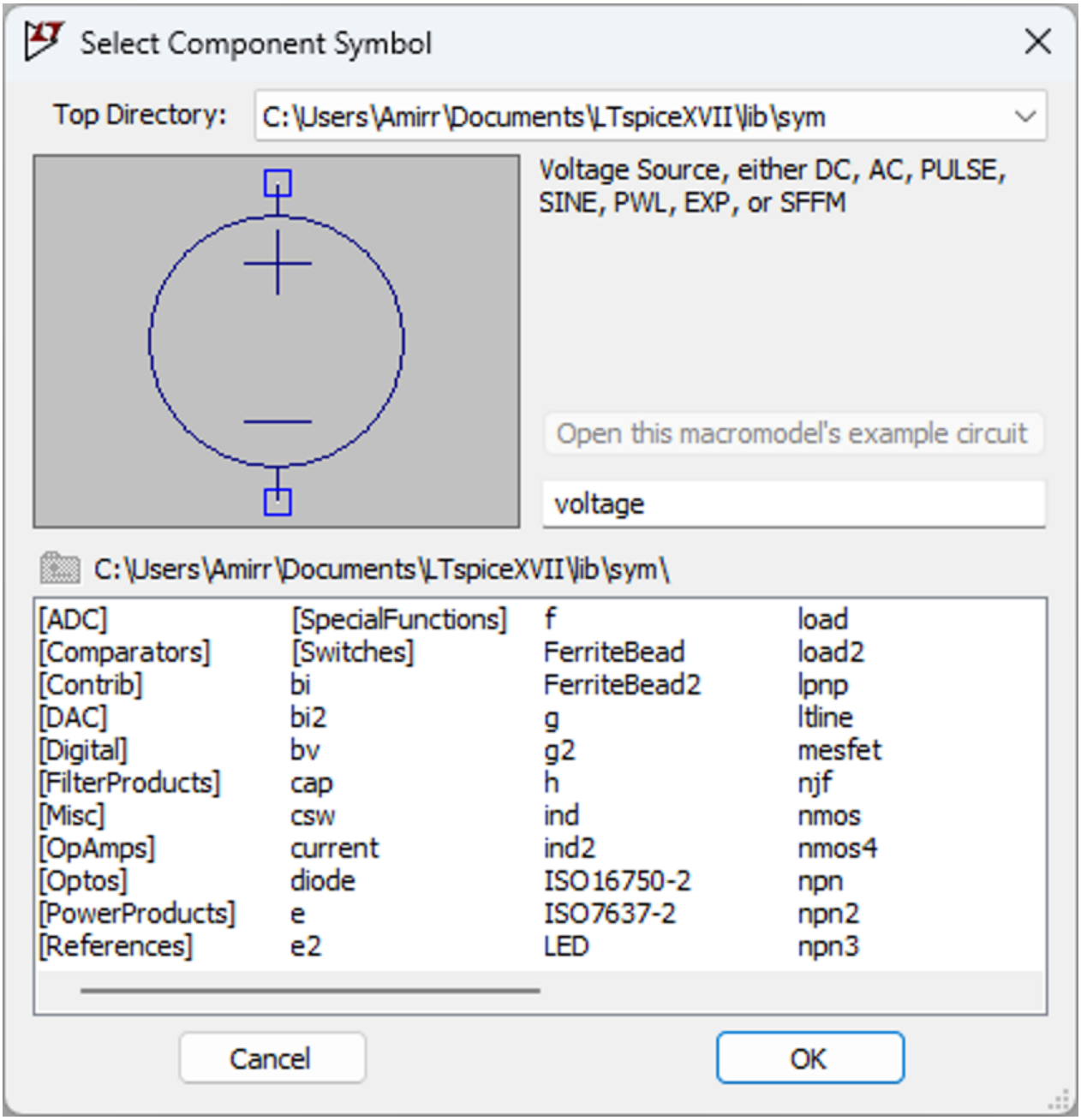


Figure 17: Diagram or recorded result: Scope, feature status, and graphical framework.

10 Signal data and plotted results

A Signal or WaveformSource object holds a sampled array and timing information such as duration and sample rate. The example signal exp(-t)u(t) on 0 to 2 seconds illustrates sampling at 2, 5, and 10 samples per second. Connecting sample points for display does not increase the numerical simulation's accuracy. Expose stop time, saved-output start time, and maximum step through the simulation interface. Voltage probes select nodes relative to ground; current and power probes select appropriate components and sign conventions. A 10-second trace sampled at 10 kHz contains 100,000 values per scalar channel. Discuss whole-array loading versus chunked processing and memory usage. Support multiple traces with different colors, legends, renaming, and color selection. Existing traces should keep stable colors when another trace is added. Evaluate expressions such as K*(V(R1+)-V(R1-)) through a defined expression parser or operator interface. Distinguish a computed sum of component signals from sequentially drawing unrelated traces. Provide horizontal/vertical grids, scale controls, and auto-fit with appropriate padding. A single cursor reports a selected coordinate/value. Two cursors report horizontal/vertical differences and slope where meaningful. Frequency measurement is not required by the stated cursor task. Preserve the original signal UML and plot examples.

```
add Rupper N001 N002 10k
add Rlower N002 N003 5k
```

Figure 18: Diagram or recorded result: Signal data and plotted results.

Table 5: Signal data and plotted results - numerical data and parameters.

Address	**0x0000**	**0x0001**	**0x0002**	**0x0003**	**...**
Value	0xab	0xcd	0xef	0xa2	...

```
{
  "circuit": {
    "coil1": {
      "turns": 100,
      "current_A": 5.2
    },
    "coil2": {
      "turns": 100,
      "current_A": 0.0
    },
    "air_gap_mm": 0.1,
    "inductance": {
      "self_inductance_H": 05.0,
      "mutual_inductance_H": 045.0,
      "coupling_coefficient": 9.0
    }
  }
}
```

Figure 19: Diagram or recorded result: Signal data and plotted results.

11 Subcircuits, schematic drawing, and node labels

Allow users to select a two-terminal one-port subcircuit, save it, and reuse it as a black-box component with two visible terminals. Its internal ground/reference policy must be explicit. A graphical rectangle can represent the saved subcircuit. The optional Thevenin/Norton reduction is restricted by the assignment to suitable linear resistive circuits; arbitrary nonlinear or dynamic circuits cannot be replaced by a constant source/resistance pair. Draw components, values, nodes, and connecting wires. Maintain a distinction between graphical position and electrical connectivity. Support the required placement, zoom/pan, and connection workflows. Labels with the same node name denote the same electrical node even when the drawing does not contain a visible wire between them. Multiple elements between the same pair of nodes must remain independently identifiable.

```
add C2 N001 N002 10u
add C1 N002 N003 1u
```

Figure 20: Diagram or recorded result: Subcircuits, schematic drawing, and node labels.

```
add L2 N001 N002 10m
add L1 N002 N003 1m
```

Figure 21: Diagram or recorded result: Subcircuits, schematic drawing, and node labels.



Figure 22: Diagram or recorded result: Subcircuits, schematic drawing, and node labels.



Figure 23: Diagram or recorded result: Subcircuits, schematic drawing, and node labels.



Figure 24: Diagram or recorded result: Subcircuits, schematic drawing, and node labels.

```
#include <cereal/archives/binary.hpp>
#include <cereal/types/string.hpp>
#include <fstream>
#include <iostream>
#include <string>

class Address {
public:
    std::string city;
    int zip;

    template <class Archive>
    void serialize(Archive& ar) {
        ar(city, zip);
    }
};

class Person {
public:
    std::string name;
    int age;
    Address address;

    template <class Archive>
    void serialize(Archive& ar) {
        ar(name, age, address);
    }
};
```

Figure 25: Implementation and output: Subcircuits, schematic drawing, and node labels.

12 AC and phase sweep controls

AC analysis uses a sinusoidal source and a frequency range with a specified number of points. The interface offers octave, decade, and linear spacing with start/stop frequency. Clearly state whether input is frequency in hertz or angular frequency in radians per second; the assignment's omega notation must not silently be interpreted as hertz. The phase-sweep interface sets base frequency, start/stop phase, and point count. Sweeping the phase of a single source in an otherwise linear time-invariant circuit does not change the magnitude of its transfer response; it shifts phase. Distinguish this input-phase experiment from plotting the transfer function's phase versus frequency. The transient menu includes stop time, saved-output start time, and maximum step. The Node Library makes all implemented phase-1 element types available for graphical placement and also exposes user-defined subcircuits. The File menu lists existing schematics and permits selection. A separate Library menu loads saved subcircuits from a library folder. The Scope menu controls result plots and the display features described above.

```
add D1 VIN GND D
add D2 VIN GND Z
```

Figure 26: Diagram or recorded result: AC and phase sweep controls.

```
int main() {
    Person p1{"Ali", 20, {"Tehran", 10001}};

    {
        std::ofstream ofs("../raw.bin", std::ios::binary);
        cereal::BinaryOutputArchive archive(ofs);
        archive(p1);
        ofs.flush();
    }

    Person p2;
    {
        std::ifstream ifs("../raw.bin", std::ios::binary);
        cereal::BinaryInputArchive archive(ifs);
        archive(p2);
    }

    std::cout << " "Name: << p2.name << "\n" Age: << p2.age
              << "\n" City: << p2.address.city << "\n" ZIP: << p2.address.zip << '\n';

    return 0;
}
```

Figure 27: Diagram or recorded result: AC and phase sweep controls.

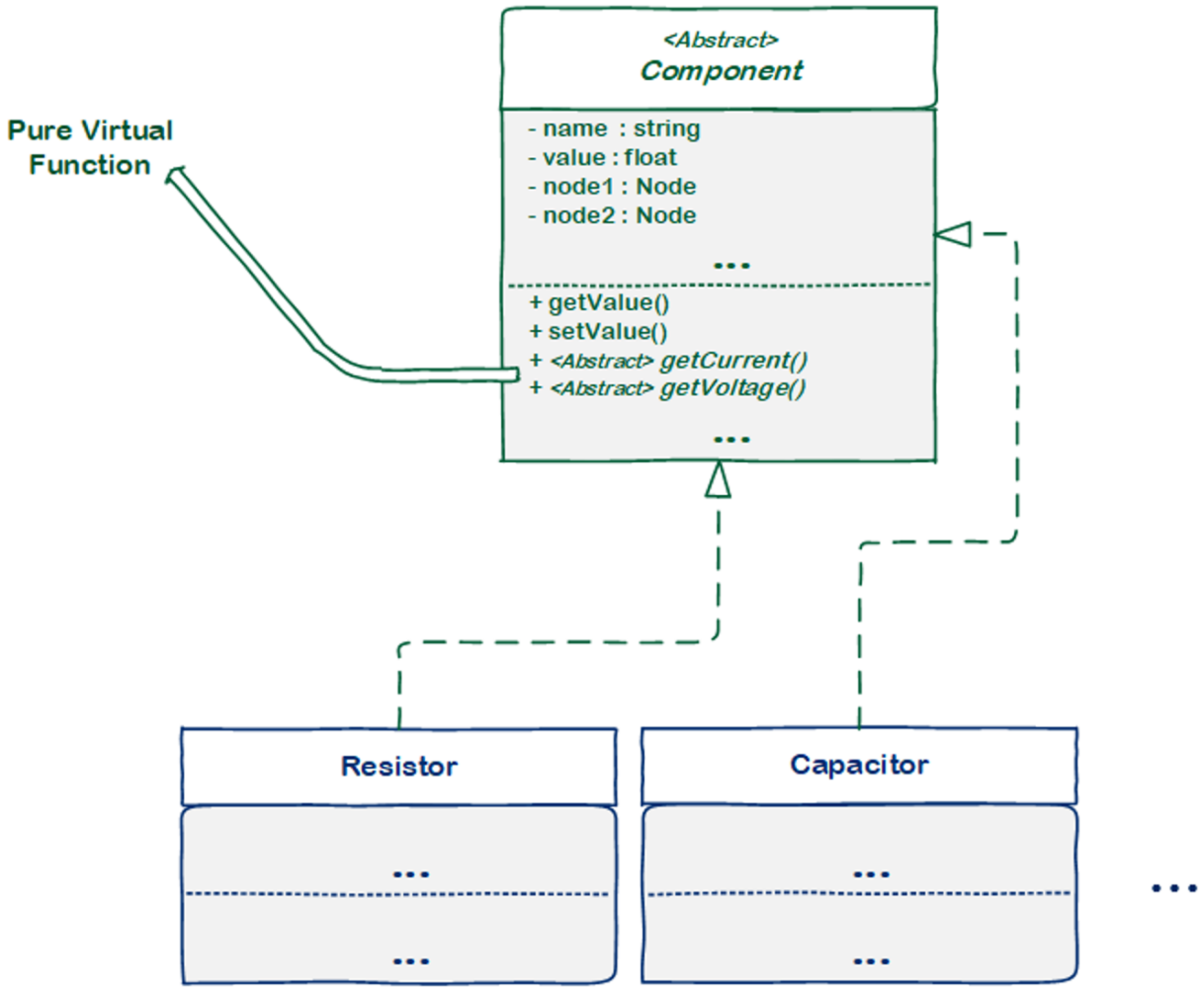


Figure 28: Diagram or recorded result: AC and phase sweep controls.

```
CEREAL_REGISTER_TYPE(Circle);
CEREAL_REGISTER_TYPE(Rectangle);
CEREAL_REGISTER_POLYMORPHIC_RELATION(Shape, Circle);
CEREAL_REGISTER_POLYMORPHIC_RELATION(Shape, Rectangle);
```

Figure 29: Diagram or recorded result: AC and phase sweep controls.

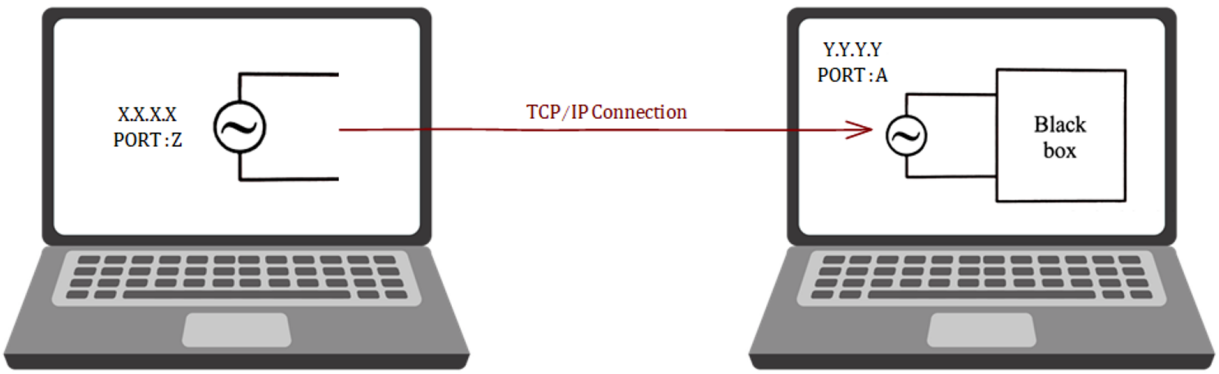


Figure 30: Diagram or recorded result: AC and phase sweep controls.

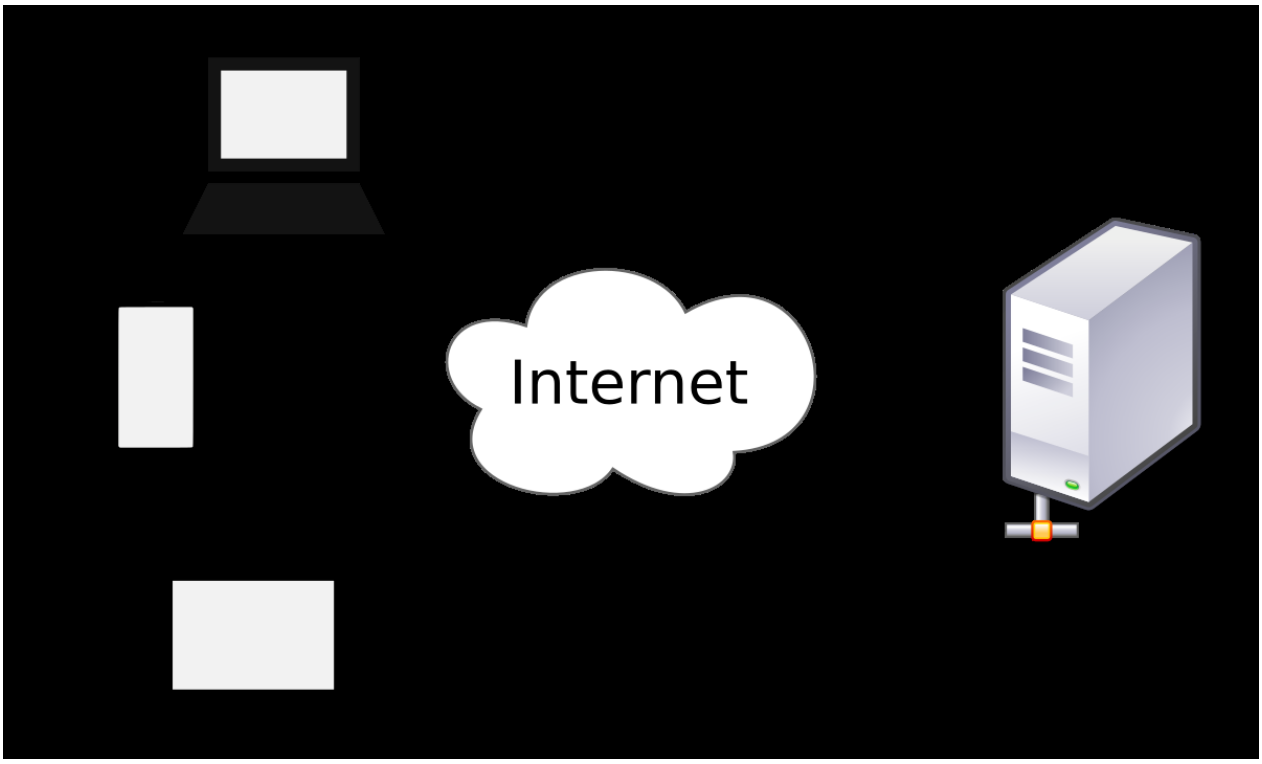


Figure 31: Diagram or recorded result: AC and phase sweep controls.

13 Project storage requirements

Save the graphical schematic, numerical circuit description, analysis settings, output signals, and user-defined subcircuits. A project folder should contain enough information to reconstruct both the drawing and the electrical analysis without manually re-entering values. Store reusable subcircuits separately in the library. Define which values are authoritative so that graphical labels and the numerical model do not diverge after loading. The source emphasizes convenient object serialization over elaborate low-level storage optimization. A generic database/storage class can provide save and load operations, but each object still needs a well-defined representation, versioning, and reference reconstruction.

```
add D1 N001 N002 D
add L1 N002 N003 1m
add D2 N003 N004 Z
```

Figure 32: Diagram or recorded result: Project storage requirements.

```
Available nodes:
n001, n002, VDD, GND, Vout, Vin
```

Figure 33: Diagram or recorded result: Project storage requirements.

```
ERROR: Invalid syntax - correct format:
.rename node <old_name> <new_name>
```

Figure 34: Diagram or recorded result: Project storage requirements.

14 Serialization and Cereal

Serialization converts selected object state into a stored representation, and deserialization reconstructs it. The original JSON example describes two coils, turns, currents, air gap, self/mutual inductance, and coupling coefficient. Explicitly list the required fields rather than copying an object's raw in-memory bytes. Pointer values, object padding, virtual-table pointers, and ABI-specific layouts are not portable persistent state. Cereal provides binary, JSON, and XML archives. The assignment chooses binary .bin files and demonstrates installing the header library from its repository. Its sample structure contains src/main.cpp, class headers/implementations, cereal/include, CMakeLists.txt, and a raw.bin result. Add the appropriate include directory to the build. Binary representation can be compact, but it is not encryption and is not guaranteed universally portable across architectures or schema versions. The example Address stores city and ZIP code. Person stores name, age, and a nested Address. Each defines a template serialize(Archive& ar) method listing its fields. BinaryOutputArchive writes the Person to an ofstream opened in binary mode; BinaryInputArchive reconstructs another Person from an ifstream. Check stream and archive errors rather than assuming that every file exists and is valid. The xxd -b raw.bin example demonstrates stored bytes, including string content and numeric fields. Cereal archives the listed fields using their serialization support; it does not generally serialize arbitrary complex objects merely by casting their addresses to char* or void*. Likewise, C++ auto performs compile-time type deduction, rather than runtime evaluation of variable types. Polymorphic component storage requires registration of derived types and relationships. The source illustrates CEREAL_REGISTER_TYPE for Circle and Rectangle and CEREAL_REGISTER_POLYMORPHIC_RELATION with Shape. Apply the corresponding pattern to the Element hierarchy and include the appropriate archive/type support. Saving through a base-class pointer without this support can fail or lose the derived representation. Preserve object identity and endpoint references when reconstructing the circuit.

```
.print TRAN <Tstep> <Tstop> [<Tstart>] [<Tmaxstep>] V(n001)
.print DC <SourceName> <StartValue> <EndValue> <Increment> V(out)
```

Figure 35: Diagram or recorded result: Serialization and Cereal.

```
.print TRAN <Tstep> <Tstop> [<Tstart>] [<Tmaxstep>] I(R1)
.print DC <SourceName> <StartValue> <EndValue> <Increment> I(C1)
```

Figure 36: Diagram or recorded result: Serialization and Cereal.

```
V V1 1 0 5
R R1 1 2 1000
R R2 2 0 2000
```

Figure 37: Diagram or recorded result: Serialization and Cereal.

```
-choose existing schematic:
1-draft1
2-draft2
3-draft3
4-elecphase1
```

Figure 38: Diagram or recorded result: Serialization and Cereal.

```
25 greater than the greatest existing node identifier
rreturn
draft1
```

Figure 39: Diagram or recorded result: Serialization and Cereal.

```
class TCPServer {
public:
    TCPServer(int port);
    ~TCPServer();
    void start();

private:
    int port;
    SOCKET serverSocket, clientSocket;
    void handleClient();
};
```

Figure 40: Implementation and output: Serialization and Cereal.

```
"ws2_32.lib") ,comment(lib #pragma

TCPServer::TCPServer(int port) {
    this >- port = port;
    this >- serverSocket = INVALID_SOCKET;
    this >- clientSocket = INVALID_SOCKET;
    WSADATA wsaData;
    WSAStartup(MAKEWORD(2, 2), &wsaData);
}

void TCPServer::start() {
    sockaddr_in serverAddr{};
    serverAddr.sin_family = AF_INET;
    serverAddr.sin_addr.s_addr = INADDR_ANY;
    serverAddr.sin_port = htons(port);

    serverSocket = socket(AF_INET, SOCK_STREAM, 0);
    bind(serverSocket, (sockaddr*)&serverAddr, sizeof(serverAddr));
    listen(serverSocket, SOMAXCONN);

    std::cout << " port on listening "Server << port << "...\n";

    sockaddr_in clientAddr{};
    int clientSize = sizeof(clientAddr);
    clientSocket = accept(serverSocket, (sockaddr*)&clientAddr, &clientSize);

    if (clientSocket != INVALID_SOCKET) {
        handleClient();
    }
}

void TCPServer::handleClient() {
    char buffer[1024] = {};
    recv(clientSocket, buffer, sizeof(buffer), 0);
    std::cout << " "Received: << buffer << "\n";

    std::string reply = server!" from "Hello;
    send(clientSocket, reply.c_str(), reply.size(), 0);
}
```

Figure 41: Diagram or recorded result: Serialization and Cereal.

15 Networking extension and its limits

The networking section is bonus work for two-person teams and required for three-person teams under the assignment. Use a server/client architecture to transfer an externally generated voltage waveform between two instances. The prescribed demonstration handles one connection at a time; concurrent multi-client service is not required. The server listens, accepts a connection, exchanges data, and closes it. The client connects, exchanges data, and closes it. IPv4 addresses are 32 bits (four bytes). IPv6 addresses are 128 bits (sixteen bytes). DNS maps domain names to address records. Ports identify transport endpoints associated with processes/services. A socket provides the operating-system interface for connection and data exchange; the protocol and application framing remain separate responsibilities. The source's Windows example uses winsock2.h, WSAStartup, AF_INET/SOCK_STREAM, bind, listen, accept, recv, send, closesocket, and the ws2_32 library. TCPServer stores its port and listening/connected socket handles. TCPClient stores the server address and port. A reusable implementation must check return values, close handles, perform WSACleanup appropriately, and handle partial send/recv operations. TCP is a byte stream: one send does not guarantee one complete matching recv. Serialize a Signal for transfer. A two-second trace at 1000 samples per second contains 2000 samples. At four bytes per float, its sample payload is 8000 bytes, excluding metadata and framing. A single 1024-byte buffer cannot receive that entire payload in one call. Define a length-prefixed message or another explicit frame format and receive until complete. Streaming chunks reduces latency and memory compared with transferring a whole long trace before playback. For the local demonstration, both computers connect to the same LAN. On Windows, ipconfig exposes the server's IPv4 interface address; supply that address and the chosen port to the client. A router's gateway address is not the server's address. Private IPv4 addresses are not restricted to 192.168.x.x; other private ranges exist. The original university address and nslookup example are historical illustrations, not verified current service endpoints. First demonstrate a transferred voltage source. The further extension transfers a serialized linear resistive subcircuit or its Thevenin/Norton equivalent, reconstructs it at the server, and connects it into the server circuit. The assignment limits this equivalent-circuit task to resistive circuits. Sending a full arbitrary dynamic circuit requires additional state and model semantics beyond this example.

```
draft2:
V1 in 0 DC 0 AC 1 SIN(0 1 1k)
R1 in out 1k
C1 out 0 1uF
tran 1ms.0 10ms
.ac dec 10 10 100k
.end
```

Figure 42: Diagram or recorded result: Networking extension and its limits.

```
-choose existing schematic:
1-draft1
2-draft2
3-draft3
4-elecphase1
```

Figure 43: Diagram or recorded result: Networking extension and its limits.

```
class TCPClient {
public:
    TCPClient(const std::string& ip, int port);
    ~TCPClient();
    void sendMessage(const std::string& message);

private:
    std::string serverIP;
    int port;
};
```

Figure 44: Implementation and output: Networking extension and its limits.

```
"ws2_32.lib") ,comment(lib #pragma

TCPClient::TCPClient(const std::string& ip, int port) {
    this >- serverIP = ip;
    this >- port = port;
    WSADATA wsaData;
    WSAStartup(MAKEWORD(2, 2), &wsaData);
}

void TCPClient::sendMessage(const std::string& message) {
    SOCKET sock = socket(AF_INET, SOCK_STREAM, 0);
    sockaddr_in serverAddr{};
    serverAddr.sin_family = AF_INET;
    serverAddr.sin_port = htons(port);
    inet_pton(AF_INET, serverIP.c_str(), &serverAddr.sin_addr);

    if (connect(sock, (sockaddr*)&serverAddr, sizeof(serverAddr)) == SOCKET_ERROR) {
        std::cerr << failed. "Connection\n";
        closesocket(sock);
        return;
    }

    send(sock, message.c_str(), message.size(), 0);
    char buffer[1024] = {};
    recv(sock, buffer, sizeof(buffer), 0);
    std::cout << " says: "Server << buffer << "\n";

    closesocket(sock);
}
```

Figure 45: Diagram or recorded result: Networking extension and its limits.

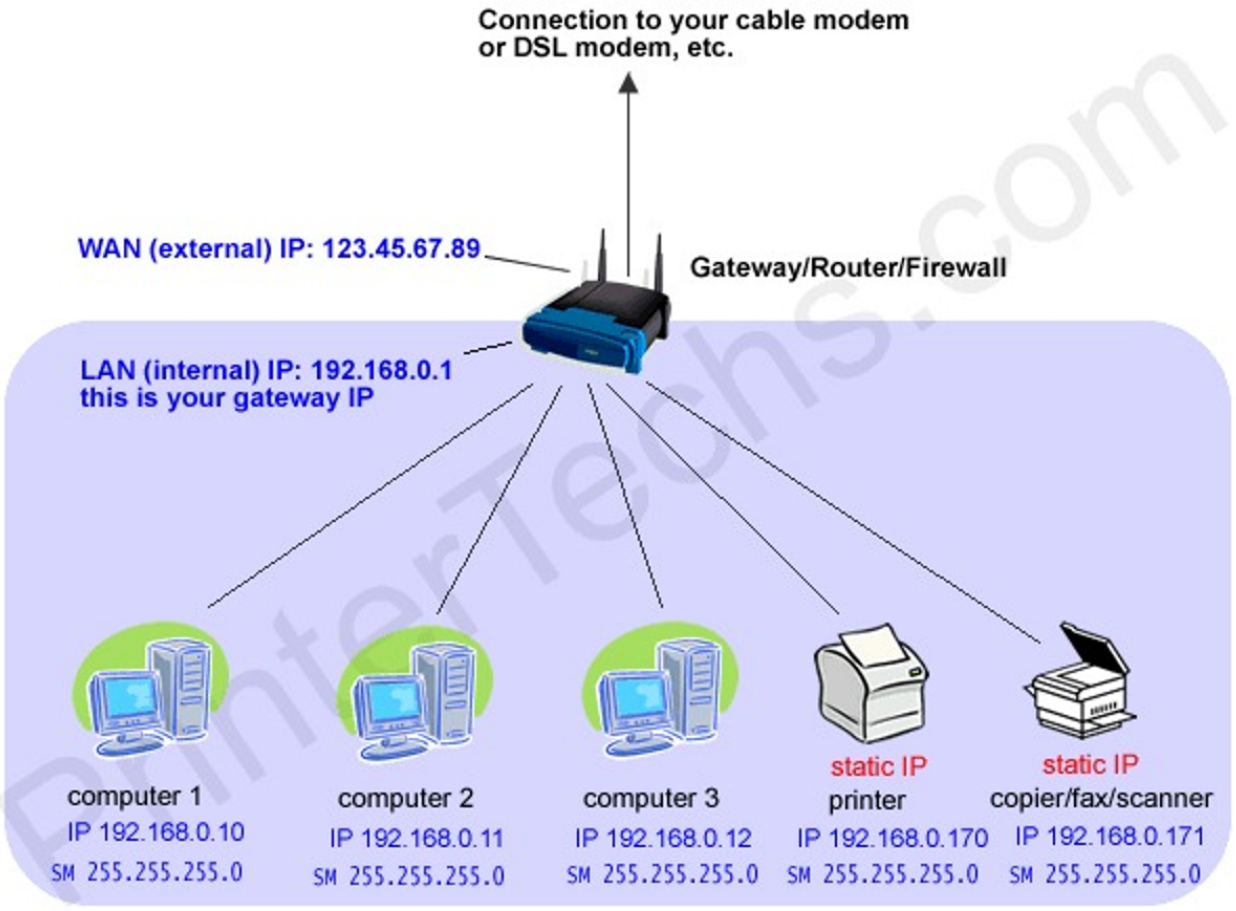


Figure 46: Diagram or recorded result: Networking extension and its limits.

```
Wireless LAN adapter Wi-Fi:

   Connection-specific DNS Suffix  . : sharif.ir
   Link-local IPv6 Address . . . . . : XXXX::XXXX:XXXX:XXXX:XXXX
   IPv4 Address. . . . . . . . . . . : X.X.X.X
   Subnet Mask . . . . . . . . . . . : 2550.240.255.
   Default Gateway . . . . . . . . . : X.X.X.X
```

Figure 47: Diagram or recorded result: Networking extension and its limits.

16 Deliverable organization

Submit the graphical source, build configuration, libraries/assets required for reproduction, saved-circuit examples, and documented demonstrations. Mark optional and unimplemented features explicitly.

Error: No ground node detected in the circuit.
Error: Negative or zero value for a component is invalid.
Error: Dependent source has an undefined control element.
Error: Missing parameters for time-dependent source.
Error: Duplicate node or component name detected.

Figure 48: Diagram or recorded result: Deliverable organization.

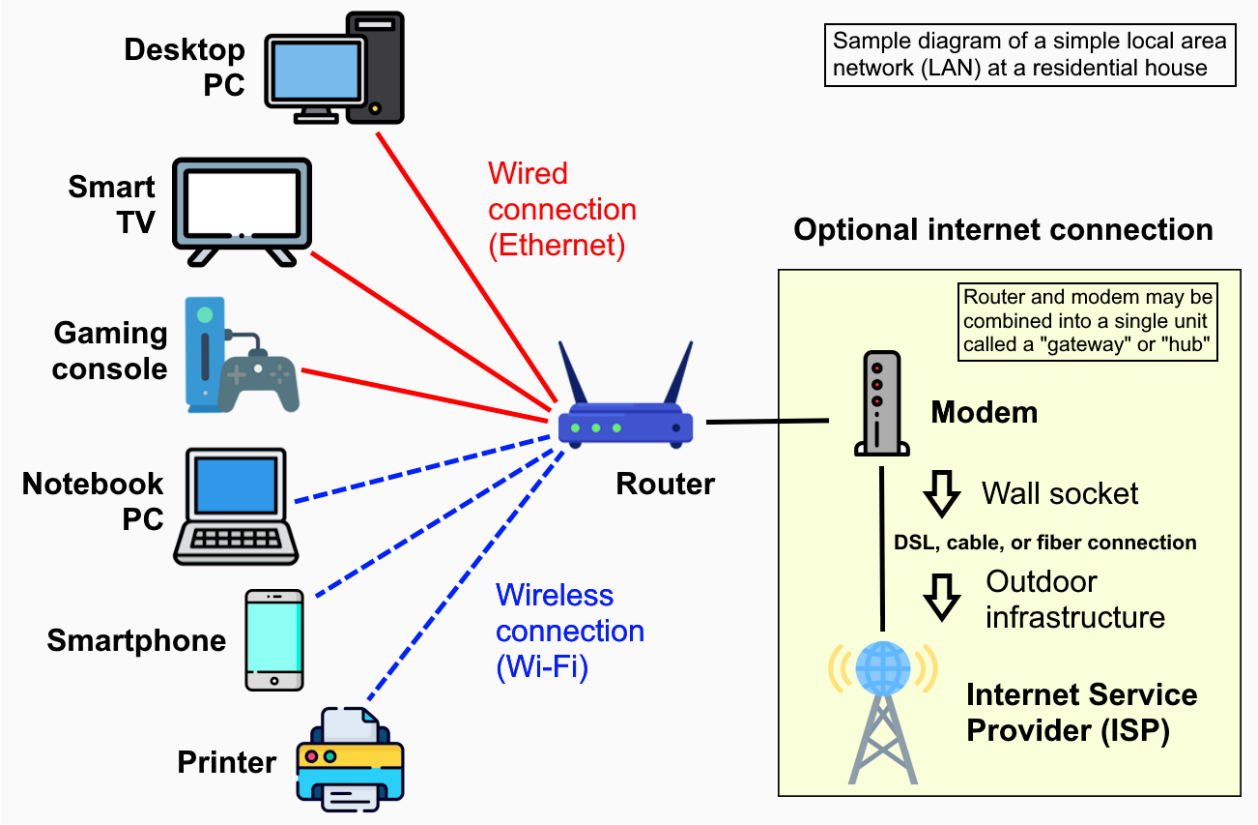


Figure 49: Diagram or recorded result: Deliverable organization.

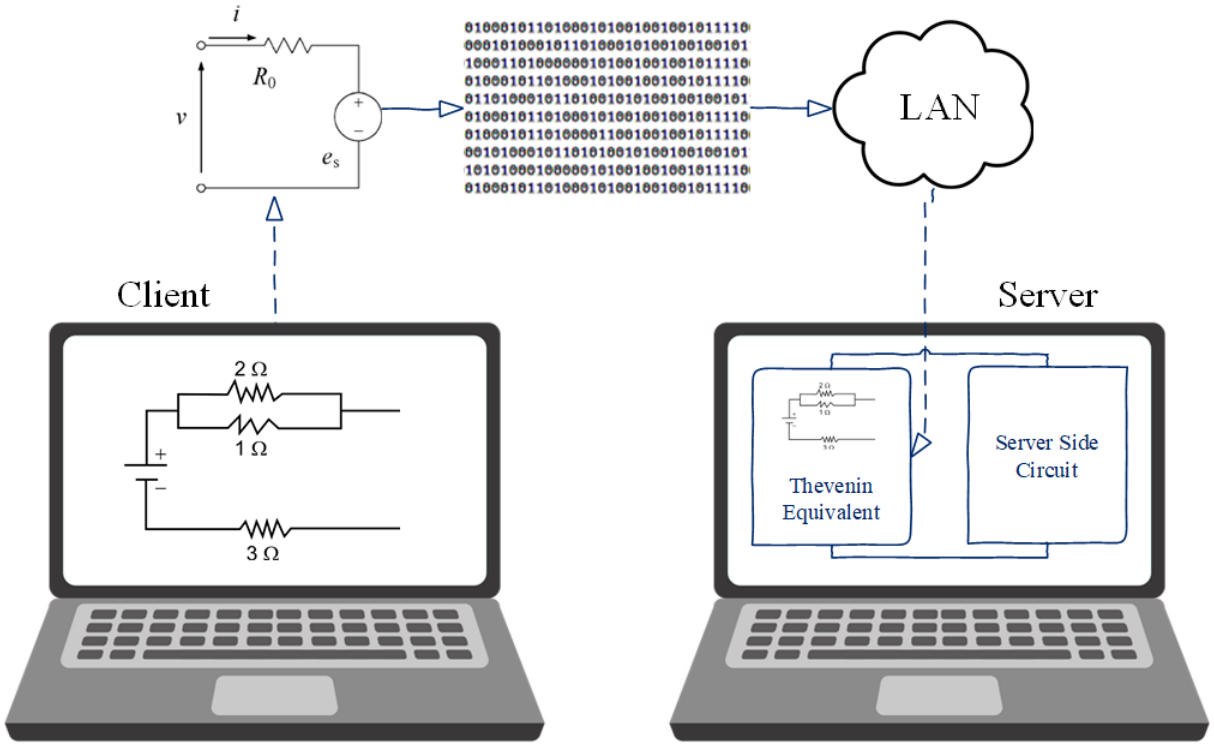


Figure 50: Diagram or recorded result: Deliverable organization.